

```

1  linux> telnet kittyhawk.cmcl.cs.cmu.edu 8000  Client: open connection
2  Trying 128.2.194.242...
3  Connected to kittyhawk.cmcl.cs.cmu.edu.
4  Escape character is '^]'.
5  GET /cgi-bin/adder?15000&213 HTTP/1.0  Client: request line
6                                         Client: empty line terminates headers
7  HTTP/1.0 200 OK                        Server: response line
8  Server: Tiny Web Server                Server: identify server
9  Content-length: 115                    Adder: expect 115 bytes in response body
10 Content-type: text/html                Adder: expect HTML in response body
11                                         Adder: empty line terminates headers
12 Welcome to add.com: THE Internet addition portal. Adder: first HTML line
13 <p>The answer is: 15000 + 213 = 15213  Adder: second HTML line in response body
14 <p>Thanks for visiting!                Adder: third HTML line in response body
15 Connection closed by foreign host.     Server: closes connection
16 linux>                                Client: closes connection and terminates

```

图 11-28 一个提供动态 HTML 内容的 HTTP 事务

旁注 将 HTTP POST 请求中的参数传递给 CGI 程序

对于 POST 请求，子进程也需要重定向标准输入到已连接描述符。然后，CGI 程序会从标准输入中读取请求主体中的参数。

 **练习题 11.5** 在 10.11 节中，我们警告过你关于在网络应用中使用 C 标准 I/O 函数的危险。然而，图 11-27 中的 CGI 程序却能没有任何问题地使用标准 I/O。为什么呢？

11.6 综合：TINY Web 服务器

我们通过开发一个虽小但功能齐全的称为 TINY 的 Web 服务器来结束对网络编程的讨论。TINY 是一个有趣的程序。在短短 250 行代码中，它结合了许多我们已经学习到的思想，例如进程控制、Unix I/O、套接字接口和 HTTP。虽然它缺乏一个实际服务器所具备的功能性、健壮性和安全性，但是它足够用来为实际的 Web 浏览器提供静态和动态的内容。我们鼓励你研究它，并且自己实现它。将一个实际的浏览器指向你自己的服务器，看着它显示一个复杂的带有文本和图片的 Web 页面，真是非常令人兴奋（甚至对我们这些作者来说，也是如此！）。

1. TINY 的主程序

图 11-29 展示了 TINY 的主程序。TINY 是一个迭代服务器，监听在命令行中传递来的端口上的连接请求。在通过调用 `open_listenfd` 函数打开一个监听套接字以后，TINY 执行典型的无限服务器循环，不断地接受连接请求（第 32 行），执行事务（第 36 行），并关闭连接的它那一端（第 37 行）。

2. doit 函数

图 11-30 中的 `doit` 函数处理一个 HTTP 事务。首先，我们读和解析请求行（第 11~14 行）。注意，我们使用图 11-8 中的 `rio_readlineb` 函数读取请求行。

TINY 只支持 GET 方法。如果客户端请求其他方法（比如 POST），我们发送给它一个错误信息，并返回到主程序（第 15~19 行），主程序随后关闭连接并等待下一个连接请求。否则，我们读并且（像我们将要看到的那样）忽略任何请求报头（第 20 行）。

code/netp/tiny/tiny.c

```
1  /*
2   * tiny.c - A simple, iterative HTTP/1.0 Web server that uses the
3   *      GET method to serve static and dynamic content
4   */
5  #include "csapp.h"
6
7  void doit(int fd);
8  void read_requesthdrs(rio_t *rp);
9  int parse_uri(char *uri, char *filename, char *cgiargs);
10 void serve_static(int fd, char *filename, int filesize);
11 void get_filetype(char *filename, char *filetype);
12 void serve_dynamic(int fd, char *filename, char *cgiargs);
13 void clienterror(int fd, char *cause, char *errnum,
14                 char *shortmsg, char *longmsg);
15
16 int main(int argc, char **argv)
17 {
18     int listenfd, connfd;
19     char hostname[MAXLINE], port[MAXLINE];
20     socklen_t clientlen;
21     struct sockaddr_storage clientaddr;
22
23     /* Check command-line args */
24     if (argc != 2) {
25         fprintf(stderr, "usage: %s <port>\n", argv[0]);
26         exit(1);
27     }
28
29     listenfd = Open_listenfd(argv[1]);
30     while (1) {
31         clientlen = sizeof(clientaddr);
32         connfd = Accept(listenfd, (SA *)&clientaddr, &clientlen);
33         Getnameinfo((SA *)&clientaddr, clientlen, hostname, MAXLINE,
34                     port, MAXLINE, 0);
35         printf("Accepted connection from (%s, %s)\n", hostname, port);
36         doit(connfd);
37         Close(connfd);
38     }
39 }
```

code/netp/tiny/tiny.c

图 11-29 TINY Web 服务器

然后，我们将 URI 解析为一个文件名和一个可能为空的 CGI 参数字符串，并且设置一个标志，表明请求的是静态内容还是动态内容（第 23 行）。如果文件在磁盘上不存在，我们立即发送一个错误信息给客户端并返回。

最后，如果请求的是静态内容，我们就验证该文件是一个普通文件，而我们是有限权限的（第 31 行）。如果是这样，我们就向客户端提供静态内容（第 36 行）。相似地，如果请求的是动态内容，我们就验证该文件是可执行文件（第 39 行），如果是这样，我们就继续，并且提供动态内容（第 44 行）。

code/netp/tiny/tiny.c

```

1 void doit(int fd)
2 {
3     int is_static;
4     struct stat sbuf;
5     char buf[MAXLINE], method[MAXLINE], uri[MAXLINE], version[MAXLINE];
6     char filename[MAXLINE], cgiargs[MAXLINE];
7     rio_t rio;
8
9     /* Read request line and headers */
10    Rio_readinitb(&rio, fd);
11    Rio_readlineb(&rio, buf, MAXLINE);
12    printf("Request headers:\n");
13    printf("%s", buf);
14    sscanf(buf, "%s %s %s", method, uri, version);
15    if (strcasecmp(method, "GET")) {
16        clienterror(fd, method, "501", "Not implemented",
17                    "Tiny does not implement this method");
18        return;
19    }
20    read_requesthdrs(&rio);
21
22    /* Parse URI from GET request */
23    is_static = parse_uri(uri, filename, cgiargs);
24    if (stat(filename, &sbuf) < 0) {
25        clienterror(fd, filename, "404", "Not found",
26                    "Tiny couldn't find this file");
27        return;
28    }
29
30    if (is_static) { /* Serve static content */
31        if (!(S_ISREG(sbuf.st_mode)) || !(S_IRUSR & sbuf.st_mode)) {
32            clienterror(fd, filename, "403", "Forbidden",
33                        "Tiny couldn't read the file");
34            return;
35        }
36        serve_static(fd, filename, sbuf.st_size);
37    }
38    else { /* Serve dynamic content */
39        if (!(S_ISREG(sbuf.st_mode)) || !(S_IXUSR & sbuf.st_mode)) {
40            clienterror(fd, filename, "403", "Forbidden",
41                        "Tiny couldn't run the CGI program");
42            return;
43        }
44        serve_dynamic(fd, filename, cgiargs);
45    }
46 }

```

code/netp/tiny/tiny.c

图 11-30 TINY doit 处理一个 HTTP 事务

3. clienterror 函数

TINY 缺乏一个实际服务器的许多错误处理特性。然而，它会检查一些明显的错误，并把它们报告给客户端。图 11-31 中的 `clienterror` 函数发送一个 HTTP 响应到客户端，在响应行中包含相应的状态码和状态消息，响应主体中包含一个 HTML 文件，向浏览器

的用户解释这个错误。

```

1 void clienterror(int fd, char *cause, char *errnum,
2                  char *shortmsg, char *longmsg)
3 {
4     char buf[MAXLINE], body[MAXBUF];
5
6     /* Build the HTTP response body */
7     sprintf(body, "<html><title>Tiny Error</title>");
8     sprintf(body, "%s<body bgcolor=\"ffffff\">\r\n", body);
9     sprintf(body, "%s%s: %s\r\n", body, errnum, shortmsg);
10    sprintf(body, "%s<p>%s: %s\r\n", body, longmsg, cause);
11    sprintf(body, "%s<hr><em>The Tiny Web server</em>\r\n", body);
12
13    /* Print the HTTP response */
14    sprintf(buf, "HTTP/1.0 %s %s\r\n", errnum, shortmsg);
15    Rio_writen(fd, buf, strlen(buf));
16    sprintf(buf, "Content-type: text/html\r\n");
17    Rio_writen(fd, buf, strlen(buf));
18    sprintf(buf, "Content-length: %d\r\n\r\n", (int)strlen(body));
19    Rio_writen(fd, buf, strlen(buf));
20    Rio_writen(fd, body, strlen(body));
21 }

```

code/netp/tiny/tiny.c

图 11-31 TINY clienterror 向客户端发送一个出错消息

回想一下，HTML 响应应该指明主体中内容的大小和类型。因此，我们选择创建 HTML 内容为一个字符串，这样一来我们可以简单地确定它的大小。还有，请注意我们为所有的输出使用的都是图 10-4 中健壮的 `rio_writen` 函数。

4. read_requesthdrs 函数

TINY 不使用请求报头中的任何信息。它仅仅调用图 11-32 中的 `read_requesthdrs` 函数来读取并忽略这些报头。注意，终止请求报头的空文本行是由回车和换行符对组成的，我们在第 6 行中检查它。

```

1 void read_requesthdrs(rio_t *rp)
2 {
3     char buf[MAXLINE];
4
5     Rio_readlineb(rp, buf, MAXLINE);
6     while(strcmp(buf, "\r\n")) {
7         Rio_readlineb(rp, buf, MAXLINE);
8         printf("%s", buf);
9     }
10    return;
11 }

```

code/netp/tiny/tiny.c

图 11-32 TINY read_requesthdrs 读取并忽略请求报头

5. parse_uri 函数

TINY 假设静态内容的主目录就是它的当前目录，而可执行文件的主目录是 `./cgi-bin`。任何包含字符串 `cgi-bin` 的 URI 都会被认为表示的是对动态内容的请求。默认的文件名是 `./home.html`。

图 11-33 中的 `parse_uri` 函数实现了这些策略。它将 URI 解析为一个文件名和一个可选的 CGI 参数字符串。如果请求的是静态内容(第 5 行)，我们将清除 CGI 参数字符串(第 6 行)，然后将 URI 转换为一个 Linux 相对路径名，例如 `./index.html`(第 7~8 行)。如果 URI 是用 “/” 结尾的(第 9 行)，我们将把默认的文件名加在后面(第 10 行)。另一方面，如果请求的是动态内容(第 13 行)，我们就会抽取所有的 CGI 参数(第 14~20 行)，并将 URI 剩下的部分转换为一个 Linux 相对文件名(第 21~22 行)。

```

code/netp/tiny/tiny.c
1  int parse_uri(char *uri, char *filename, char *cgiargs)
2  {
3      char *ptr;
4
5      if (!strstr(uri, "cgi-bin")) { /* Static content */
6          strcpy(cgiargs, "");
7          strcpy(filename, ".");
8          strcat(filename, uri);
9          if (uri[strlen(uri)-1] == '/')
10             strcat(filename, "home.html");
11         return 1;
12     }
13     else { /* Dynamic content */
14         ptr = index(uri, '?');
15         if (ptr) {
16             strcpy(cgiargs, ptr+1);
17             *ptr = '\0';
18         }
19         else
20             strcpy(cgiargs, "");
21         strcpy(filename, ".");
22         strcat(filename, uri);
23         return 0;
24     }
25 }

```

code/netp/tiny/tiny.c

图 11-33 TINY `parse_uri` 解析一个 HTTP URI

6. serve_static 函数

TINY 提供五种常见类型的静态内容：HTML 文件、无格式的文本文件，以及编码为 GIF、PNG 和 JPG 格式的图片。

图 11-34 中的 `serve_static` 函数发送一个 HTTP 响应，其主体包含一个本地文件的内容。首先，我们通过检查文件名的后缀来判断文件类型(第 7 行)，并且发送响应行和响应报头给客户端(第 8~13 行)。注意用一个空行终止报头。

```

code/netp/tiny/tiny.c
1 void serve_static(int fd, char *filename, int filesize)
2 {
3     int srcfd;
4     char *srcp, filetype[MAXLINE], buf[MAXBUF];
5
6     /* Send response headers to client */
7     get_filetype(filename, filetype);
8     sprintf(buf, "HTTP/1.0 200 OK\r\n");
9     sprintf(buf, "%sServer: Tiny Web Server\r\n", buf);
10    sprintf(buf, "%sConnection: close\r\n", buf);
11    sprintf(buf, "%sContent-length: %d\r\n", buf, filesize);
12    sprintf(buf, "%sContent-type: %s\r\n\r\n", buf, filetype);
13    Rio_writen(fd, buf, strlen(buf));
14    printf("Response headers:\n");
15    printf("%s", buf);
16
17    /* Send response body to client */
18    srcfd = Open(filename, O_RDONLY, 0);
19    srcp = Mmap(0, filesize, PROT_READ, MAP_PRIVATE, srcfd, 0);
20    Close(srcfd);
21    Rio_writen(fd, srcp, filesize);
22    Munmap(srcp, filesize);
23 }
24
25 /*
26  * get_filetype - Derive file type from filename
27  */
28 void get_filetype(char *filename, char *filetype)
29 {
30     if (strstr(filename, ".html"))
31         strcpy(filetype, "text/html");
32     else if (strstr(filename, ".gif"))
33         strcpy(filetype, "image/gif");
34     else if (strstr(filename, ".png"))
35         strcpy(filetype, "image/png");
36     else if (strstr(filename, ".jpg"))
37         strcpy(filetype, "image/jpeg");
38     else
39         strcpy(filetype, "text/plain");
40 }
code/netp/tiny/tiny.c

```

图 11-34 TINY serve_static 为客户端提供静态内容

接着，我们将被请求文件的内容复制到已连接描述符 `fd` 来发送响应主体。这里的代码是比较微妙的，需要仔细研究。第 18 行以读方式打开 `filename`，并获得它的描述符。在第 19 行，Linux `mmap` 函数将被请求文件映射到一个虚拟内存空间。回想我们在第 9.8 节中对 `mmap` 的讨论，调用 `mmap` 将文件 `srcfd` 的前 `filesize` 个字节映射到一个从地址 `srcp` 开始的私有只读虚拟内存区域。

一旦将文件映射到内存,就不再需要它的描述符了,所以我们关闭这个文件(第 20 行)。执行这项任务失败将导致潜在的致命的内存泄漏。第 21 行执行的是到客户端的实际文件传送。`rio_writen` 函数复制从 `srcp` 位置开始的 `filesize` 个字节(它们当然已经被映射到了所请求的文件)到客户端的已连接描述符。最后,第 22 行释放了映射的虚拟内存区域。这对于避免潜在的致命的内存泄漏是很重要的。

7. `serve_dynamic` 函数

TINY 通过派生一个子进程并在子进程的上下文中运行一个 CGI 程序,来提供各种类型的动态内容。

图 11-35 中的 `serve_dynamic` 函数一开始就向客户端发送一个表明成功的响应行,同时还包括带有信息的 Server 报头。CGI 程序负责发送响应的剩余部分。注意,这并不像我们可能希望的那样健壮,因为它没有考虑到 CGI 程序会遇到某些错误的可能性。

```

1 void serve_dynamic(int fd, char *filename, char *cgiargs)
2 {
3     char buf[MAXLINE], *emptylist[] = { NULL };
4
5     /* Return first part of HTTP response */
6     sprintf(buf, "HTTP/1.0 200 OK\r\n");
7     Rio_writen(fd, buf, strlen(buf));
8     sprintf(buf, "Server: Tiny Web Server\r\n");
9     Rio_writen(fd, buf, strlen(buf));
10
11     if (Fork() == 0) { /* Child */
12         /* Real server would set all CGI vars here */
13         setenv("QUERY_STRING", cgiargs, 1);
14         Dup2(fd, STDOUT_FILENO); /* Redirect stdout to client */
15         Execve(filename, emptylist, environ); /* Run CGI program */
16     }
17     Wait(NULL); /* Parent waits for and reaps child */
18 }

```

code/netp/tiny/tiny.c

code/netp/tiny/tiny.c

图 11-35 TINY `serve_dynamic` 为客户端提供动态内容

在发送了响应的第一部分后,我们会派生一个新的子进程(第 11 行)。子进程用来自请求 URI 的 CGI 参数初始化 `QUERY_STRING` 环境变量(第 13 行)。注意,一个真正的服务器还会在此处设置其他的 CGI 环境变量。为了简短,我们省略了这一步。

接下来,子进程重定向它的标准输出到已连接文件描述符(第 14 行),然后加载并运行 CGI 程序(第 15 行)。因为 CGI 程序运行在子进程的上下文中,它能够访问所有在调用 `execve` 函数之前就存在的打开文件和环境变量。因此,CGI 程序写到标准输出上的任何东西都将直接送到客户端进程,不会受到任何来自父进程的干涉。其间,父进程阻塞在对 `wait` 的调用中,等待当子进程终止的时候,回收操作系统分配给子进程的资源(第 17 行)。

旁注 处理过早关闭的连接

尽管一个 Web 服务器的基本功能非常简单,但是我们不想给你一个假象,以为编写一个实际的 Web 服务器是非常简单的。构造一个长时间运行而不崩溃的健壮的 Web 服务器是一件困难的任务,比起在这里我们已经学习了的内容,它要求对 Linux 系统编程有更加深入的

理解。例如，如果一个服务器写一个已经被客户端关闭了的连接(比如，因为你在浏览器上单击了“Stop”按钮)，那么第一次这样的写会正常返回，但是第二次写就会引起发送 SIGPIPE 信号，这个信号的默认行为就是终止这个进程。如果捕获或者忽略 SIGPIPE 信号，那么第二次写操作会返回值 -1，并将 `errno` 设置为 `EPIPE`。`strerr` 和 `perror` 函数将 `EPIPE` 错误报告为“Broken pipe”，这是一个迷惑了很多人的不太直观的信息。总的来说，一个健壮的服务器必须捕获这些 SIGPIPE 信号，并且检查 `write` 函数调用是否有 `EPIPE` 错误。

11.7 小结

每个网络应用都是基于客户端-服务器模型的。根据这个模型，一个应用是由一个服务器和一个或多个客户端组成的。服务器管理资源，以某种方式操作资源，为它的客户端提供服务。客户端-服务器模型中的基本操作是客户端-服务器事务，它是由客户端请求和跟随其后的服务器响应组成的。

客户端和服务端通过因特网这个全球网络来通信。从程序员的角度来看，我们可以把因特网看成是一个全球范围的主机集合，具有以下几个属性：1)每个因特网主机都有一个唯一的 32 位名字，称为它的 IP 地址。2)IP 地址的集合被映射为一个因特网域名的集合。3)不同因特网主机上的进程能够通过连接互相通信。

客户端和服务端通过使用套接字接口建立连接。一个套接字是连接的一个端点，连接以文件描述符的形式提供给应用程序。套接字接口提供了打开和关闭套接字描述符的函数。客户端和服务端通过读写这些描述符来实现彼此间的通信。

Web 服务器使用 HTTP 协议和它们的客户端(例如浏览器)彼此通信。浏览器向服务器请求静态或者动态的内容。对静态内容的请求是通过从服务器磁盘取得文件并把它返回给客户端来服务的。对动态内容的请求是通过在服务器上一个子进程的上下文中运行一个程序并将它的输出返回给客户端来服务的。CGI 标准提供了一组规则，来管理客户端如何将程序参数传递给服务器，服务器如何将参数以及其他信息传递给子进程，以及子进程如何将它的输出发送回客户端。只用几百行 C 代码就能实现一个简单但是有功效的 Web 服务器，它既可以提供静态内容，也可以提供动态内容。

参考文献说明

有关因特网的官方信息源被保存在一系列的可免费获取的带编号的文档中，称为 RFC(Requests for Comments，请求注解，Internet 标准(草案))。在以下网站可获得可搜索的 RFC 的索引：

<http://rfc-editor.org>

RFC 通常是因特网基础设施的开发者编写的，因此，对于普通读者来说，往往过于详细了。然而，要想获得权威信息，没有比它更好的信息来源了。HTTP/1.1 协议记录在 RFC 2616 中。MIME 类型的权威列表保存在：

<http://www.iana.org/assignments/media-types>

Kerrisk 是全面 Linux 编程的圣经，提供了现代网络编程的详细讨论[62]。关于计算机网络互联有大量很好的通用文献[65, 84, 114]。伟大的科技作家 W. Richard Stevens 编写了一系列相关的经典文献，如高级 Unix 编程[111]、因特网协议[109, 120, 107]，以及 Unix 网络编程[108, 110]。认真学习 Unix 系统编程的学生会想要研究所有这些内容。不幸的是，Stevens 在 1999 年 9 月 1 日逝世。我们会永远记住他的贡献。

家庭作业

- 11.6 A. 修改 TINY 使得它会原样返回每个请求行和请求报头。

B. 使用你喜欢的浏览器向 TINY 发送一个对静态内容的请求。把 TINY 的输出记录到一个文件中。

C. 检查 TINY 的输出，确定你的浏览器使用的 HTTP 的版本。

D. 参考 RFC 2616 中的 HTTP/1.1 标准，确定你的浏览器的 HTTP 请求中每个报头的含义。你可以从 www.rfc-editor.org/rfc.html 获得 RFC 2616。
- 11.7 扩展 TINY，使得它可以提供 MPG 视频文件。用一个真正的浏览器来检验你的工作。